

# Autour des expressions régulières

Les objectifs de ce TP sont les suivants :

- Comprendre les entrées/sorties des flux de données (notamment en terminal), et savoir les rediriger. Utiliser des *pipes*.
- Connaître le rôle des arguments de la fonction `main` : `int main(int argc, char* argv[])`, et savoir utiliser des arguments depuis la ligne de commande.
- Apprendre à lire et écrire dans un fichier en C.
- Manipuler les expressions régulières.

Les trois sections de ce TP sont relativement indépendantes, **les trois sont au programme et à savoir utiliser après rappel!**

## A Excursion dans le terminal

### A.1 grep et regex concrètes

Un fichier `francais.txt` est fourni avec ce TP. Il contient un dictionnaire français assez complet (323422 formes), avec les flexions (pluriel, féminin, formes conjuguées des verbes...), à raison d'un mot par ligne. Les caractères ont été convertis en ASCII « standard » pour simplifier (il n'y a plus d'accents, de cédilles...), et les traits d'union ont été supprimés ( `timbreposte` au lieu de *timbre-poste* ).

L'utilitaire `grep` permet de rechercher des occurrences d'une expression régulière dans un fichier. Une occurrence d'une expression régulière est une suite de caractères formant un mot (au sens mathématique du terme) appartenant au langage de l'expression : on utilise souvent l'anglicisme *match*.

- L'utilisation de base est `grep [options] pattern [file]`. Si aucun fichier n'est précisé, la recherche est faite sur l'entrée standard.
- `grep` travaille ligne par ligne : une occurrence à cheval sur deux lignes n'est pas considérée.
- Par défaut, `grep` affiche chaque ligne dans laquelle il y a (au moins) une occurrence.

Exemple :

Recherche de "fif" dans un dictionnaire français contenant un mot par ligne. Seules les lignes contenant au moins une occurrence sont affichées, et les occurrences sont mises en valeur à l'intérieur de chaque ligne (la ligne `fiftyfifty` contient deux occurrences).

```
> grep 'fif' francais.txt
fifille
fifilles
fifre
fifrelin
fifrelin
fifres
fifres
fiftyfifty
rififi
rififis
sousfifre
sousfifres
```

- Les deux seules options que nous utiliserons ici sont :
  - `-E` qui active le mode « expressions régulières étendues » ;
  - `-o` qui fait que `grep` n'affiche que les occurrences, et non les lignes contenant des occurrences.

On utilisera **systématiquement** l'option `-E`, et parfois l'option `-o`.

*Exemple :* La commande `grep -E 'a.*e.*i.*o.*u' francais.txt` permet de chercher les mots de la langue française qui contiennent toutes les voyelles simultanément (sauf y), dans cet ordre (d'abord a puis e puis i etc.).

1. Tapez cette commande dans le terminal (à l'endroit où se trouve `francais.txt`) et observez le résultat.

#### Astuces :

- Le point `.` remplace n'importe quelle caractère, comme le symbole  $\Sigma$  du cours. Ainsi, pour désigner le  $\Sigma^*$  du cours, on utilise `.*`.
  -  Notez que `grep` cherche **une occurrence** de l'expression régulière dans la ligne. Ainsi, si `[expr]` est une expression régulière, il revient au même de rechercher l'expression `[expr]` ou l'expression `.*[expr].*`.
  - N'hésitez pas à vous référer au cours sur la syntaxe concrète des expressions régulières.
2. À l'aide de la commande `grep`, affichez tous les mots :
    - a. qui contiennent au moins un « w » et au moins un « q ».
    - b. qui contiennent au moins 6 fois la lettre « i ».
    - c. qui contiennent au moins 10 voyelles.
    - d. qui commencent par un « p » et contiennent au moins 9 voyelles?

## A.2 Pipes

L'utilisation des *pipes* en ligne de commande **est au programme** (sans rappel).

#### Entrées et sorties :

Dans le terminal comme dans un fichier, nous manipulons les données via des canaux d'entrée/sortie. Vous en avez déjà entendu parler l'année dernière (TP algorithme de Huffman.)

#### Rappel :

Pour lire/écrire dans un fichier, comme pour lire/écrire dans un terminal, on manipule des canaux (*channel* en Anglais) sur lesquels on "envoie" ou "récupère" notre texte.

Le canal de sortie standard qui permet d'écrire sur le terminal est prédéfini et se nomme `stdout` (ainsi que `stderr` pour les erreurs). Les fonctions d'affichage usuelles (comme `printf` en C ou `print_string` en OCaml) utilisent le canal `stdout` par défaut, sans qu'on ait besoin de le préciser.

De même, la lecture passe par des canaux d'entrée d'où on "extraie" du texte. Pour le terminal, le canal standard prédéfini se nomme `stdin`.

La plupart des utilitaires en ligne de commande prennent leur entrée sur l'entrée standard et émettent leur résultat sur la sortie standard, ou permettent en tout cas de le faire. Pour `grep`, par exemple :

- le résultat est émis sur la sortie standard ;
  - l'entrée peut être un fichier (ex : `grep -E 'q[a-z]*q' blah.txt`), mais sera l'entrée standard si aucun fichier n'est précisé.
3. Tapez la commande `grep -E 'a'` dans un terminal, puis entrez un texte et observez. (Utilisez `ctrl-c` pour sortir.)

## Redirections

Il est possible de **rediriger** l'entrée ou la sortie standard d'une commande avec les symboles `<` et `>` :

- `<input` permet de préciser que l'on prend l'entrée dans le fichier nommé `input` à la place de l'entrée standard. Par exemple, `grep -E 'beaux' <francais.txt` prend son entrée dans le fichier `francais.txt`. Dans le cas de `grep`, ça n'a pas d'intérêt puisqu'on peut simplement lui préciser le fichier en argument, sans avoir besoin de redirection : `grep -E 'beaux' francais.txt` est une commande équivalente.
- `>output` permet d'écrire le résultat de la commande sur le fichier nommé `output` au lieu de le faire sur la sortie standard. Si le fichier n'existe pas, il est créé. Par exemple, `echo` est une commande permettant de répéter l'entrée dans la sortie. `echo hello world` affichera simplement `hello world` dans votre terminal. `echo hello world >output.txt` écrira "hello world" dans le fichier `output.txt`.

4. Tapez la commande `grep -E 'beaux' francais.txt >resultat.txt` dans le terminal et observez **attentivement** le résultat de la commande. Que se passe-t-il ? Expliquer pourquoi rien ne s'affiche.

Imaginons maintenant que l'on souhaite compter le nombre de lignes du fichier `moby_dick.txt` contenant (au moins) une occurrence de `wallow` .

- Un appel à `grep -E 'wallow' moby_dick.txt` extrait les lignes qui nous intéressent et les envoie sur la sortie standard.
- D'autre part, l'utilitaire `wc` (pour *Word Count*) permet de compter le nombre d'octets, de mots et de lignes d'un fichier. Comme seul le nombre de lignes nous intéresse ici, on pourra utiliser `wc --lines` .
- Il y aura donc deux étapes : un appel à `grep` , avec `moby_dick.txt` comme entrée, puis un appel à `wc` sur le résultat.

## Pipes

Une première solution est de rediriger la sortie de `grep` vers un fichier temporaire, d'appeler `wc` sur ce fichier, puis de le supprimer :

Terminal

```
1 $ grep -E 'wallow' moby_dick.txt >tmp.txt
2 $ wc --lines tmp.txt
3 35 tmp.txt
4 $ rm tmp.txt
```

Cependant, l'utilitaire `wc` peut tout à fait utiliser l'entrée standard comme entrée au lieu d'un fichier. Une *meilleure* solution serait donc de rediriger la sortie standard de `grep` **vers l'entrée standard de `wc`** . C'est possible grâce à l'opérateur `|` (que l'on lit *pipe*, pour « tuyau » ou « tube ») :

Terminal

```
1 $ grep -E 'wallow' moby_dick.txt | wc --lines
2 35
```

### Remarque :

- Le résultat est un peu différent parce que `wc` affiche le nom du fichier d'entrée quand il y en a un, mais cela n'a aucune importance.

5. En utilisant des *pipes*, trouver :

- Combien il y a de lignes contenant au moins une occurrence de `the` dans `moby_dick.txt` .
- Combien il y a de lignes de `moby_dick.txt` contenant au moins deux occurrences de `the` .

6. Que calcule la commande `grep -E -o '(\W|^)the(\W|$)' moby_dick.txt | wc --lines` ?

7. Afficher les mots contenant au moins 8 voyelles dont au moins 2 fois la lettre 'y' dans `francais.txt`.

*Indication : on peut enchaîner les greps avec des pipes, n'essayez pas de simuler une intersection dans une seule expression régulière.*

## B Arguments de la fonction main

Jusqu'à maintenant, nous avons toujours écrit des fonctions `main` qui ne prennent pas d'argument. Nous écrivons alors :

C

```
1 int main(void) { //avec ou sans le "void"
2     ...
3 }
```

En réalité, la fonction `main` peut prendre des arguments en entrées : on les lui passe **depuis la ligne de commande**.

Par exemple, si à la compilation on crée un exécutable nommé `exec` , il est possible de passer en argument la chaîne de caractères "bonjour" à notre fonction `main` en écrivant `./exec bonjour` . On peut donner autant d'arguments que l'on souhaite à la fonction `main` , en les séparant par des espaces. Ainsi, `./exec hello world` donnera deux arguments à la fonction `main` : la chaîne de caractères "hello" et la chaîne de caractères "world".

Pour récupérer les arguments de la fonction `main`, on écrit `int main(int argc, char *argv[])`. `argc` contient le nombre d'arguments passés en ligne de commande et `argv` est un tableau de chaînes de caractères de longueur `argc` qui contient chacun des arguments donnés. Par convention, on nomme toujours les arguments de la fonction `main` **argv** et **argc**.

#### Remarques :

- ⚠ Le premier argument, `argv[0]`, est **toujours** le nom du programme. Dans l'exemple ci-dessus, on a `argv[0] = "./exec"`. Ce n'est que `argv[1]` qui contient "bonjour". En particulier, le tableau `argv` n'est jamais vide.

**Exemple basique :** Lecture des arguments passés en ligne de commande.

```
1 int main(int argc, char *argv[]) {
2     for (int i = 0; i < argc; i++) {
3         printf("argv[%d]=%s\n", i, argv[i]);
4     }
5 }
```

- Créer un fichier C contenant cette seule fonction `main`. Compilez le et lancez l'exécutable en lui donnant des arguments de votre choix. Faites varier le nombre et la longueur des arguments.

⚠ Tous les arguments donnés sont des chaînes de caractères. Si vous tapez `./exec 42`, vous obtiendrez en argument la chaîne de caractères "42" et non l'entier 42. Pour convertir une chaîne de caractères numérique en entier (`int`), on utilise la fonction `atoi` : `int atoi(const char *nptr)`. On peut aussi convertir en long ou en long long avec `atol` et `atoll`.

Par exemple, `int n = atoi("42");` met l'entier 42 dans la variable `n`. `atoi` renvoie 0 si la chaîne de caractère donnée ne commence pas par une valeur numérique.

- Écrire une fonction qui prend en ligne de commande un argument entier, et affiche dans le terminal le double de cet entier.

*Avant d'accéder à `argv[1]`, on vérifiera qu'on a donné au moins 1 argument en ligne de commande, en plus de la commande.*

## C Lecture et écriture dans un fichier en C

Vous avez vu l'année dernière comment lire et écrire dans un fichier en OCaml (cf TP algorithme d'Huffman). Nous allons maintenant voir comment lire et écrire dans un fichier en C.

Comme en OCaml, pour lire et écrire dans un fichier, on commence par accéder aux canaux de lecture et d'écriture dans ce fichier. Pour ouvrir ces canaux, on utilise la fonction `fopen` de prototype :

```
FILE *fopen(const char *pathname, const char *mode) .
```

Cette fonction prend en entrée d'abord le nom du fichier (ou un chemin si le fichier n'est pas dans le répertoire courant), puis le mode d'ouverture du fichier, et renvoie un pointeur vers le canal du fichier qu'on a demandé d'ouvrir. Par exemple, `fopen("monfichier.txt", "w")` ouvre un canal vers le fichier `monfichier.txt`, pour écrire sur le fichier. Au programme de MP2I/MPI, il n'y a que les deux modes suivants à connaître :

- `w` (pour *write*) : mode d'écriture.
- `r` (pour *read*) : mode de lecture.

(HP) : Il existe un mode `a` (pour *append*) qui permet d'ajouter du texte à un fichier sans effacer son contenu déjà présent.

Pour écrire ou lire dans un fichier, il suffit d'écrire dans le canal ouvert sur ce fichier. On utilise des variantes des fonctions `printf` et `scanf` qui prennent en compte l'entrée/sortie spécifiée, et s'utilisent de la même façon : `fprintf` et `fscanf`. La seule différence est qu'il faut leur donner un pointeur vers un fichier (canal de sortie/entrée) en premier argument.

- `fprintf` écrit la chaîne de caractères donnée dans le canal de sortie précisé en argument. Par exemple :

```
fprintf(fp, "La réponse est %d.\n", 42)
```

écrit la phrase "La réponse est 42.\n" dans le canal de fichier `fp` donné (en mode "w").

- `fscanf` lit la chaîne de caractères formatée donnée. Par exemple, si on sait que la ligne actuelle du fichier contient 3 entiers séparés par des espaces (comme 42 53 -21), on peut les récupérer dans trois entiers `n1`, `n2` et `n3` par :  
`fscanf(fp, "%d %d %d", &n1, &n2, &n3);`  
(en mode "r").

**Rappel :** Lorsqu'on écrit des chaînes de caractères dans un canal, les caractères s'accumulent dans le canal (dans ce qu'on appelle un *buffer*) sans être immédiatement écrits sur la destination. Lorsque le canal est suffisamment plein, il est vidé d'un seul coup dans le fichier/terminal destination. Ainsi, en cas d'erreur ou d'interruption prématurée, rien n'apparaîtra sur le terminal/fichier même si votre commande `(f)printf` a bien été exécutée. De même, l'affichage peut arriver plus tard que ce à quoi vous vous attendez. Pour provoquer l'écriture/affichage immédiatement, il faut *flush* le canal, avec la fonction `int fflush(FILE *stream)`.

**⚠ ATTENTION**, il ne faut jamais oublier de fermer un canal que vous avez ouvert<sup>1</sup> ! En particulier parce que votre OS ne peut ouvrir qu'un nombre limité de fichiers simultanément, mais aussi parce que la fermeture du fichier va automatiquement *flush* le canal. La fonction permettant de fermer un canal en C est `int fclose(FILE *stream)`. On lui donne simplement en argument le canal à fermer. La fonction renvoie 0 si tout s'est bien passé, ou un code d'erreur sinon.

## Exemples basiques :

- Voici un exemple de fonction qui écrit un message dans un fichier :

**C : fichiers.c**

```
1 void ecrit(const char* fichier, const char* message) {
2     FILE* fp = fopen(fichier, "w");
3     fprintf(fp, "%s", message);
4     fclose(fp);
5 }
```

Exemple d'utilisation :

**C : fichiers.c**

```
1 int main(void){
2     ecrit("test.txt", "Hello_world!\n");
3     return EXIT_SUCCESS;
4 }
```

10. À l'aide des deux petits codes C fournis ci-dessus, créez un fichier `test.txt` qui contient la chaîne de caractères `Hello World!\n`.

*Ne cherchez pas loin, il suffit vraiment de lancer les codes donnés ci-dessus.*

### Remarques :

- Si le fichier donné en argument n'existe pas, il est créé lors du processus d'écriture.
- **⚠** on écrit SUR le fichier donné. S'il contenant déjà du texte, celui-ci sera écrasé par votre écriture (on peut par contre écrire plusieurs lignes dans le fichier à la suite, l'emplacement d'écriture se déplace au fur et à mesure qu'on écrit). On peut déplacer nous même la tête d'écriture pour éviter d'effacer un texte préalable, mais ceci est hors programme.
- En réalité, pour manipuler nos fichiers proprement, il faudrait s'assurer qu'on a bien réussi à ouvrir un canal avant de le manipuler. Il suffit de vérifier que le pointeur `fp` obtenu n'est pas NULL. (S'il l'est, en général on renvoie une erreur). Pour simplifier, on ne le fera pas dans ce TP.
- Pour lire un fichier, c'est un petit peu plus délicat. Il nous faut disposer d'une chaîne de caractères (de type `char*`, donc) dans laquelle nous allons stocker ce que nous lisons. Une chaîne de caractères n'est rien de plus qu'un tableau de caractères en C, et sa taille est donc fixée avant d'écrire dedans.  
Nous allons définir un nombre maximal de caractères qu'il est possible de lire, par exemple 10000 caractères. Si une ligne (ou autre) dépasse cette taille, elle ne sera pas lue correctement. On crée un tableau de caractères de cette taille. Ensuite, on peut récupérer une ligne (ou autre) de notre fichier dans cette chaîne de caractères via `fscanf`.

**Exemple : lecture de la première ligne d'un fichier.** (On l'affiche dans le terminal.)

1. Comme souvent, en réalité, votre OS le fera pour vous à la fin de l'exécution de votre programme... mais prenez l'habitude !

C : fichiers.c

```

1 void lit_premiere_ligne(const char* fichier) {
2     FILE* fptr = fopen(fichier, "r");
3     char chaine[10000] = ""; // Chaîne vide de taille 10000, elle contient donc uniquement
4     //le caractère '\0'
5
6     // On lit jusqu'à rencontrer le caractère "\n", ou maximum 10000 caractères du fichier,
7     //et on stocke le tout dans "chaine" :
8     fscanf(fptr, "%[^\n]", chaine);
9
10    printf("%s\n", chaine);
11    fclose(fptr);
12 }

```

- Ainsi, on peut lire le fichier ligne par ligne comme on le faisait en OCaml, ou mot par mot avec un simple `fscanf(fptr, "%s", chaine)` à la place, ou de plein d'autres manières encore.

**Exemple : lecture du fichier ligne par ligne :**

C : fichiers.c

```

1 void lit(const char* fichier) {
2     FILE* fptr = fopen(fichier, "r");
3     char chaine[10000] = "";
4
5     //scanf renvoie le nombre d'items matchés avec succès, ou un code d'erreur.
6     //EOF (End Of File) est renvoyé en particulier lorsqu'on essaie de lire sur la fin du fichier
7     //On lit jusqu'à ce que fscanf nous renvoie ce code d'erreur.
8     while (fscanf(fptr, "%[^\n]\n", chaine) != EOF) {
9         printf("%s\n", chaine);
10    }
11    fclose(fptr);
12 }

```

11. Vérifiez que vous avez bien tout compris en lisant via C le contenu du fichier "message.txt" et en affichant son contenu dans le terminal.

#### Entraînements :

12. Écrire un programme C (dans un nouveau fichier) qui prend en argument un nom de fichier en ligne de commande et affiche le nombre de lignes<sup>2</sup> contenues dans ce fichier. **N'utilisez aucun copier-coller**, et ne recopiez pas bêtement les codes ci-dessus. Essayez de les comprendre dans un premier temps, puis de les écrire par vous-mêmes sans le code sous les yeux.
13. En utilisant la lecture dans un fichier en C, trouvez la première ligne dans laquelle apparaît le mot "wallow" dans le fichier "moby\_dick.txt". Lorsque vous avez trouvé, levez la main et faites moi confirmer votre ligne.  
*Indication : vous pouvez utiliser les fonctions `strncmp` et `strlen` pour vous faciliter un peu la vie, de la bibliothèque `<string.h>`. Allez voir la doc pour leur utilisation !*  
*Aucune contrainte de complexité n'est attendue ici, une recherche naïve suffira. Attention par contre à bien fermer le fichier dans tous les cas.*

On veut maintenant coder une recherche de mot dans un texte de manière plus efficace, en améliorant la recherche naïve avec l'**algorithme de Rabin-Karp**.

#### Rappels (algorithme de Rabin-Karp) :

L'idée est d'utiliser une **empreinte tournante**  $h$  (*rolling hash* en anglais). Plutôt que de tester à chaque position  $i$  du texte si le mot est présent à l'indice  $i$ , on teste d'abord en temps constant si l'empreinte du mot coïncide avec l'empreinte de cette partie du texte.

- Si l'empreinte est différente (majorité des cas), on passe directement à la position suivante.
- Sinon, on regarde si le mot est présent à l'indice  $i$ .

Pour que le test d'égalité des empreintes soit calculable en temps constant  $O(1)$ , il faut que l'empreinte de  $t[i+1:j+1]$  se calcule en temps constant à partir de l'empreinte de  $t[i:j]$ .

Prenons pour ce TP la fonction de hash suivante :  $h(t[i:j]) = t[i] + t[i+1] + \dots + t[j-1]$ , la somme des lettres qui composent le mot (vues comme des entiers : leur encodage ASCII).

2. Oui, nous sommes en train de recoder wc.

14. En appliquant l'algorithme de Rabin-Karp sur chaque ligne, écrire une fonction :

`bool chercheRK(const char* fichier, const char* mot)` qui renvoie `true` si le mot est présent dans une des lignes du fichier et faux sinon.

Testez votre algorithme sur le fichier `moby_dick` en y cherchant les mots suivants : "whale", "dolphin", "goldfish", "jellyfish", "shark", "sea horse" et "turtle".